

National University of Computer and Emerging Sciences



AL-2002: Artificial Intelligence Lab Manual 06

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives.....	3
1 Hill Climbing Algorithm.....	3

Objectives

After performing this lab, students shall be able to understand implementation of the following :

- ✓ Hill Climbing Algorithm

1 Hill Climbing Algorithm

Local search algorithms are a class of search techniques that focus on finding an optimal solution by iteratively improving a single candidate solution. Unlike traditional search algorithms that explore a tree or graph and maintain a record of all explored paths, local search algorithms maintain only the current state in memory. These algorithms are particularly useful for optimization problems where the goal is to find the best possible state rather than the exact path to that state. Hill Climbing is one of the simplest forms of local search algorithms and is widely used for problems with large state spaces.

Hill Climbing is a greedy local search algorithm that continuously moves in the direction of increasing value (or decreasing cost).

It is analogous to: *A person climbing a hill in fog, taking steps that go uphill, hoping to reach the peak.*

Hill Climbing evaluates only the neighboring states of the current position and moves to the one that appears to be the best. If no neighbor improves the current state, the algorithm stops, potentially at a local maximum rather than the global optimum.

In hill climbing:

- A state represents a complete candidate solution
- Each state has a set of neighboring states
- Each state is assigned a value by an evaluation function

Example:

- State: position on a hill
- Neighbors: nearby positions
- Value: height of the hill

The evaluation function measures the quality of a state.

- Denoted as: $f(\text{state})$
- Higher value means better state (for maximization problems)
- Lower value means better state (for minimization problems)

Algorithm Steps

- Choose an initial state
- Loop:
 - Evaluate current state
 - Generate neighbors
 - Select the best neighbor
 - If best neighbor is better:
 - Move to that neighbor
 - Else:
 - Stop (local optimum reached)

While simple and fast, this approach has inherent limitations, such as getting stuck at local maxima, plateaus, or ridges, where the immediate neighbors do not offer improvement even though a better solution exists elsewhere.

There are several variants of Hill Climbing.

- Simple Hill Climbing examines neighbors one by one and moves to the first better neighbor found, which makes it prone to missing better alternatives.
- Steepest-Ascent Hill Climbing evaluates all neighbors and moves to the best among them, improving accuracy but requiring more computation.
- Stochastic Hill Climbing introduces randomness by selecting a random neighbor from those that improve the current state, which helps in escaping some local maxima and adds diversity to the search.

One of the main challenges of Hill Climbing is that it is not complete; it does not guarantee a solution exists for all problems. The algorithm can easily get stuck at a local maximum, a plateau, or a ridge. A local maximum occurs when a state has a higher value than all its neighbors but is not the global maximum. Plateaus are flat regions where all neighboring states have the same value, providing no direction for improvement. Ridges are narrow paths to the global maximum that cannot be reached through purely greedy moves. Due to these limitations, Hill Climbing is generally not optimal and may require enhancements to reliably reach global optima.

To address the limitations of basic Hill Climbing, several techniques can be used. **Random Restart Hill Climbing** involves running the algorithm multiple times from different initial states and selecting the best solution obtained. Sideways moves allow the algorithm to traverse plateaus by moving to neighbors with equal evaluation values. Stochastic Hill Climbing adds randomness in selecting neighbors, which helps avoid local maxima. Adjusting the neighborhood structure or increasing the neighborhood size can also improve the algorithm's ability to explore the state space effectively.